

Evaluation Report

Chef-Kitchen final quality audit - requirement coverage, accessibility, reliability, maintainability, and frontend best practices.

Coverage Verdict

The project covers the required feature set: validated form behavior, localStorage persistence, MealDB API and JSON data usage, select controls and dialogs, CSS animation, responsive layout, URL parameter handling, and modular JavaScript organization. URL parameters handle both valid and invalid ids with clear UI states. Accessibility and UX include a native contact dialog, focus restoration, hamburger aria-expanded state, row-specific grocery delete labels, and a wired Meal Planner search input. The Meal Planner does not overflow at a 390px mobile viewport. Production build passed with npm run build, and targeted browser checks passed with no console errors.

Requirement	Status	Evidence
Validated form	Covered	Native required/email validation works; contact dialog opens only after valid input.
Local storage persistence	Covered	Meal plan persisted after reload; grocery list generated from stored plan.
API/json data usage	Covered	TheMealDB fetch helpers and local JSON imports are used.
Drop-down menu or modal	Covered	Meal Finder selects, Meal Planner native dialogs, and Contact native dialog are present.
CSS animation	Covered	Animations are present across page and component interactions.
Responsive design	Covered	Mobile audit at 390px showed no Meal Planner page overflow.
Accessibility and UX	Covered	Native dialog, focus return, nav state, delete labels, and planner search are present.
URL parameters	Covered	Valid id loads recipe; invalid id shows a Recipe not found state without undefined text.
Modular code	Covered	Shared template helpers are separate from Meal Finder page behavior.

Top 12 Findings

HIGH

1. Dynamic HTML rendering still accepts API/localStorage content

Why it matters: Several views interpolate recipe, ingredient, and grocery data into HTML. Dynamic rendering can expose XSS if API data or localStorage is poisoned.

Evidence: templates.js:43, mealfinder.js:88, mealplanner.js:183, grocerylist.js:20

Concrete fix: Replace dynamic innerHTML blocks with createElement/textContent rendering, or escape every interpolated value before insertion.

MEDIUM

2. Corrupt localStorage can still crash planner or grocery pages

Why it matters: Happy-path persistence works, but JSON.parse(localStorage.getItem(...)) remains unguarded. One malformed stored value can break page initialization.

Evidence: mealplanner.js:76, mealplanner.js:335, grocerylist.js:11

Concrete fix: Add a safeParseStorage(key, fallback) helper and validate stored objects before rendering them.

MEDIUM

3. API failures mostly disappear into console logs

Why it matters: Failed searches can leave stale or blank UI with no retry message; homepage image loading still has no per-image fallback.

Evidence: templates.js:16, mealfinder.js:258, home.js:80

Concrete fix: Add loading, empty, and error UI states with retry buttons and image fallbacks.

MEDIUM

4. Category links and search endpoint do not match

Why it matters: Homepage cards are labeled as categories but link to hard-coded recipe IDs, and filter/search logic relies on search.php?s=, which searches meal names rather than categories.

Evidence: index.html:46, mealfinder.js:234, recipedata.js:5

Concrete fix: Add proper filter.php?c= and filter.php?a= API helpers, or relabel those cards as featured recipes.

MEDIUM

5. Week and budget inputs are visible but unused

Why it matters: Users can set them, but nothing reads, validates, stores, or applies them. This makes the budget-oriented planner promise feel incomplete.

Evidence: mealplanner.html:34; no JS usage found

Concrete fix: Persist week/budget with the meal plan and apply budget logic, or remove the controls until they are functional.

MEDIUM

6. Contact form always reports success

Why it matters: The contact flow validates inputs and uses an accessible dialog, but submission is simulated with success forced to true. Users receive a confirmation even though no message is actually sent.

Evidence: contact.js:31, contact.js:33

Concrete fix: Connect the form to a real endpoint or clearly label it as a demo; handle network failures with a visible failure state.

LOW

7. Motion has no reduced-motion fallback

Why it matters: Animations and transitions are unconditional, which can hurt vestibular-sensitive users.

Evidence: mealfinder.css:180, contact.css:177, main.css:12

Concrete fix: Add @media (prefers-reduced-motion: reduce) to disable animation and transition.

LOW

8. Footer FAQ link is a placeholder

Why it matters: The footer includes an href="#" FAQ link. It appears interactive but does not take the user to useful content.

Evidence: home.js:48

Concrete fix: Link to a real FAQ page/section or remove the placeholder until the page exists.

LOW

9. Grocery checkbox state is not persisted

Why it matters: The grocery list persists generated items, but purchased/checked state is lost after reload. This weakens the grocery workflow for real shopping use.

Evidence: grocerylist.js:20

Concrete fix: Store checked state per grocery item and update localStorage when checkboxes change.

LOW

10. Grocery rendering reparses the table on every row

Why it matters: tbody.innerHTML += inside a loop repeatedly reparses accumulated HTML and rebuilds prior rows, which is inefficient and makes event handling more fragile.

Evidence: grocerylist.js:20

Concrete fix: Build rows with a DocumentFragment or map once and assign innerHTML once after escaping values.

LOW

11. Page modules still auto-run on import

Why it matters: Page modules call their init functions at module load. That is workable for pages, but it makes isolated testing and reuse harder.

Evidence: mealfinder.js:281, mealplanner.js:352

Concrete fix: Use small page entry files that call exported init functions after DOM readiness.

LOW

12. Performance quick wins remain

Why it matters: Homepage makes sequential image API calls, ships a 5.5 MB autoplay video, and bundles large JSON into route chunks.

Evidence: home.js:80; build output showed showing-food.mp4 at about 5.5 MB

Concrete fix: Use Promise.allSettled, cache API results, compress/lazy-load video, and load long descriptions only when needed.

Ship-Now Checklist

Must-fix before shipping:

- Fix unsafe dynamic HTML rendering across API/localStorage views.
- Guard and validate localStorage parsing.
- Add visible API error/loading/empty states.
- Wire or remove week and budget inputs.
- Align category labels with category API behavior.
- Connect contact form submission to a real endpoint or label it as demo-only.

Nice-to-have:

- Add reduced-motion CSS.
- Replace the placeholder FAQ link.
- Persist grocery purchased checkbox state.
- Use fragment-based grocery rendering.
- Move page auto-run init calls into entry files.
- Optimize sequential API calls, video weight, and large JSON loading.